

# Datamak Workflow: From Equilibrium Ingestion to Surrogate Training

Updated: 2026-03-26

## Contents

|          |                                                                                      |          |
|----------|--------------------------------------------------------------------------------------|----------|
| <b>1</b> | <b>Purpose</b>                                                                       | <b>2</b> |
| <b>2</b> | <b>Central Database and Core Tables</b>                                              | <b>2</b> |
| 2.1      | Key tables . . . . .                                                                 | 2        |
| 2.2      | Operational status contracts . . . . .                                               | 2        |
| <b>3</b> | <b>Workflow Overview</b>                                                             | <b>3</b> |
| 3.1      | Step 0: Bootstrap the schema . . . . .                                               | 3        |
| 3.2      | Step 1: Ingest equilibria into <code>data_equil</code> . . . . .                     | 3        |
| 3.2.1    | Mate Kinetic EFIT path . . . . .                                                     | 3        |
| 3.2.2    | TRANSP path . . . . .                                                                | 3        |
| 3.3      | Step 2: Create studies that connect equilibria to a GK code . . . . .                | 4        |
| 3.4      | Step 3: Generate <code>gk_input</code> rows and input content . . . . .              | 4        |
| 3.4.1    | Pfile/Gfile-driven input generation . . . . .                                        | 4        |
| 3.4.2    | TRANSP-driven input generation . . . . .                                             | 4        |
| 3.5      | Step 4: Backfill parsed physics columns from <code>gk_input.content</code> . . . . . | 5        |
| 3.6      | Step 5: Promote selected inputs to execution . . . . .                               | 5        |
| 3.7      | Step 6: Batch export and remote execution . . . . .                                  | 5        |
| 3.7.1    | Create a batch database . . . . .                                                    | 5        |
| 3.7.2    | Deploy and execute remotely . . . . .                                                | 5        |
| 3.7.3    | Sync back to the main database . . . . .                                             | 6        |
| <b>4</b> | <b>Surrogate Training Workflow</b>                                                   | <b>6</b> |
| 4.1      | Training data selection . . . . .                                                    | 6        |
| 4.2      | Feature construction . . . . .                                                       | 6        |
| 4.3      | Target definition . . . . .                                                          | 6        |
| 4.4      | Model fitting and artifacts . . . . .                                                | 7        |
| 4.5      | Registering and using a trained surrogate . . . . .                                  | 7        |
| <b>5</b> | <b>End-to-End Summary</b>                                                            | <b>7</b> |
| <b>6</b> | <b>Practical Command Reference</b>                                                   | <b>8</b> |
| <b>7</b> | <b>Current Limitations</b>                                                           | <b>8</b> |

# 1 Purpose

This document describes the main Datamak workflow from equilibrium ingestion to surrogate-model training. It is intended as a bridge between the high-level architecture notes and the concrete scripts used in daily operation.

The scope is the path

`data_equil → gk_study → gk_input → gk_run → gk_surrogate`

with emphasis on the repository entry points that implement each step.

## 2 Central Database and Core Tables

The main SQLite database is `gyrokinetic_simulations.db` at repository root. The baseline schema is created by `database/create_gyrokinetic_db.py`.

### 2.1 Key tables

| Table                     | Role                                                                                                                                          |
|---------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------|
| <code>data_origin</code>  | Metadata describing where equilibria came from.                                                                                               |
| <code>data_equil</code>   | Raw equilibrium/profile inputs, including stored <code>pfile</code> , <code>gfile</code> , or <code>transpfile</code> references and content. |
| <code>gk_code</code>      | Gyrokinetic code identity such as GX or CGYRO.                                                                                                |
| <code>gk_model</code>     | Active input templates and model flags used to generate GK inputs.                                                                            |
| <code>gk_study</code>     | A study linking one equilibrium to one gyrokinetic code.                                                                                      |
| <code>gk_input</code>     | Generated gyrokinetic inputs, their content, radial coordinate, parsed physics columns, and workflow status.                                  |
| <code>gk_batch</code>     | Metadata for exported remote batch databases.                                                                                                 |
| <code>gk_run</code>       | Remote execution records and output metrics such as <code>gamma_max</code> , <code>ky_abs_mean</code> , and <code>diffusion</code> .          |
| <code>gk_surrogate</code> | Registry of trained surrogate artifacts and metadata.                                                                                         |
| <code>sg_estimate</code>  | Surrogate predictions attached to existing <code>gk_input</code> rows.                                                                        |

### 2.2 Operational status contracts

Datamak uses status fields as workflow contracts between CLI scripts, the GUI, and remote execution.

- `gk_input.status`: typically `NEW`, `WAIT`, `TORUN`, `BATCH`, `CRASHED`, `SUCCESS`, `ERROR`.
- `gk_batch.status`: typically `CREATED`, `SENT`, `LAUNCHED`, `SYNCED`.
- `gk_run.status`: workflow-dependent, commonly `TORUN`, `RUNNING`, `SUCCESS`, `CONVERGED`, `CRASHED`.

## 3 Workflow Overview

### 3.1 Step 0: Bootstrap the schema

The repository bootstrap script `database/create_gyrokinetic_db.py` creates the main schema and applies guarded additive upgrades, including:

- `data_origin` and `data_equil` for source data,
- `gk_study`, `gk_model`, and `gk_input` for GK setup,
- `gk_batch` and `gk_run` for execution tracking,
- supporting indexes and compatibility `ALTER TABLE` logic.

Typical command:

```
python3 database/create_gyrokinetic_db.py --db gyrokinetic_simulations.db
```

### 3.2 Step 1: Ingest equilibria into `data_equil`

There are two main ingestion families.

#### 3.2.1 Mate Kinetic EFIT path

The script `db_update/populate_data_equil_from_Mate_KinEFIT.py`:

1. scans a root directory for subfolders,
2. finds `p*` and `g*` files,
3. reads file contents into memory,
4. creates or reuses a `data_origin` row,
5. inserts new `data_equil` rows,
6. runs `update_activate_Mate_KinEFIT.sql` to activate selected rows.

In this path, the database stores both filenames and raw text content of the `pfile` and `gfile`. This makes later GK input generation self-contained.

#### 3.2.2 TRANSP path

The TRANSP-based workflows populate `data_equil` using CDF metadata instead of local `pfile/gfile` pairs.

- Semi-automatic path: `db_update/populate_data_equil_from_Alexei_Transp_09.py` and related scripts.
- Flux full-auto path: `db_update/Transp_full_auto/MainSteps_1_launch_on_laptop.sh`, `MainSteps_2_launch_on_flux.sh`, and `MainSteps_3_launch_on_laptop.sh`.

The Flux full-auto path uses a temporary Flux-local database, populates `data_equil` and `transp_timeseries` there, generates inputs remotely, then syncs the results back into the main database.

### 3.3 Step 2: Create studies that connect equilibria to a GK code

`gk_study` is the bridge between one equilibrium and one gyrokinetic code. The later input-generation scripts assume that:

- the relevant `data_equil` rows are active,
- the appropriate `gk_code` row exists,
- one or more `gk_model` rows are marked active.

In practice, studies may be created by SQL activation scripts or by the full-auto TRANSP workflow.

### 3.4 Step 3: Generate `gk_input` rows and input content

The next stage converts equilibrium data plus template settings into explicit gyrokinetic inputs.

#### 3.4.1 Pfile/Gfile-driven input generation

The script `db_update/create_gk_input_from_pyrokinetic_with_pfile_and_gfile.py` operates on active Mate equilibria. Its main responsibilities are:

1. fetch active `gk_study` rows and their stored `pfile/gfile` content,
2. fetch active `gk_model` template definitions,
3. sweep over a `psin` grid,
4. call Pyrokinetics with the requested template and local-geometry settings,
5. insert or update `gk_input` rows with generated input content.

The inserted rows include:

- `gk_study_id`,
- `gk_model_id`,
- filename and output path,
- full input content,
- `psin`,
- workflow `status`, usually starting at `WAIT`.

#### 3.4.2 TRANSP-driven input generation

The script `db_update/create_gk_input_from_pyrokinetic_with_transpfile.py` plays the same role for TRANSP data. Instead of local `pfile/gfile` pairs, it loads the TRANSP source, uses Pyrokinetics to evaluate a requested time slice, and writes the resulting GX-style input content into `gk_input.content`.

The Flux full-auto workflow performs this generation remotely, then syncs the resulting rows back into the main database.

### 3.5 Step 4: Backfill parsed physics columns from `gk_input.content`

After input content exists, Datamak extracts structured scalar features from that text using `db_update/backfill_gk_input_physics.py`.

This script parses the following sections from each input:

- `[geometry]` for quantities such as `Rmaj`, `qinp`, `shat`, `akappa`, `tri`, and `betaprim`,
- `[physics]` for scalar physics settings,
- `[species]` for electron and main-ion density, temperature, mass, charge, gradients, and collisionality.

The extracted values populate columns on `gk_input`. This step is important because the surrogate workflow reads engineered features from columns, not by reparsing raw input text at training time.

### 3.6 Step 5: Promote selected inputs to execution

Generated inputs do not directly become surrogate-training data. They first need simulation output.

The usual progression is:

1. create `gk_input` rows with status `WAIT`,
2. promote selected rows to `TORUN`,
3. export those rows to a batch database,
4. run them remotely,
5. sync `gk_run` results back into the main database.

### 3.7 Step 6: Batch export and remote execution

The execution workflow is centered on the `batch/` folder.

#### 3.7.1 Create a batch database

`batch/create_batch_database.py -copy-torun:`

- selects `gk_input.status = TORUN`,
- copies those cases into a timestamped batch database under `batch/new/`,
- creates `gk_run` rows in that batch database,
- updates source `gk_input.status` to `BATCH`,
- inserts a `gk_batch` row with status `CREATED`.

#### 3.7.2 Deploy and execute remotely

`batch/deploy_batch_large.py` transfers non-empty batch databases and HPC helper scripts to Perlmutter, updates `gk_batch` status through `SENT` and `LAUNCHED`, and submits remote jobs.

On the remote side, the job scripts consume `gk_run` rows, write input files, execute the solver, and update `gk_run.status` together with quantities such as `gamma_max`, `ky_abs_mean`, and `diffusion`.

### 3.7.3 Sync back to the main database

`batch/check_launched_batches.py` and `batch/monitor_remote_runs.py` synchronize the remote execution results and produce health reports. At this point, the main database contains the joined information needed for surrogate training:

- input-side features on `gk_input`,
- target outputs on `gk_run`.

## 4 Surrogate Training Workflow

### 4.1 Training data selection

The surrogate training script is `db_surrogate/train_gamma_surrogate.py`. It loads rows from a join of `gk_run` and `gk_input`.

By default it keeps only:

- rows whose `gk_run.status` is `SUCCESS` or `CONVERGED`,
- rows with finite target values,
- rows where all required input features can be assembled.

Optional filters exist for `data_origin` name or identifier.

### 4.2 Feature construction

The current feature vector is:

`temp_ratio`, `dens_ratio`, `electron_tprim`, `electron_fprim`, `ion_tprim`, `ion_fprim`,  
`ion_vnewk`, `electron_vnewk`, `mass_ratio`, `Rmaj`, `qinp`, `shat`, `shift`, `akappa`,  
`akappri`, `tri`, `tripri`, `betaprim`

The three ratio features are computed at training time from the stored `gk_input` columns:

- `temp_ratio = ion_temp / electron_temp`,
- `dens_ratio = ion_dens / electron_dens`,
- `mass_ratio = ion_mass / (electron_mass * 1836)`.

### 4.3 Target definition

The default regression target is `gk_run.gamma_max`. The script can also train against other `gk_run.*` quantities through the `-mapsto` argument, for example `gk_run.ky_abs_mean` or `gk_run.diffusion`.

## 4.4 Model fitting and artifacts

The current baseline model is a `sklearn.ensemble.RandomForestRegressor`. The script supports:

- optional train/test splitting,
- configurable tree count and leaf size,
- optional `log1p` transform of the target.

Outputs are written to `db_surrogate/models/` by default:

- a pickle payload containing the fitted model and metadata,
- a JSON sidecar with non-binary metadata such as feature ranges, metrics, status filter, and target mapping.

Representative command:

```
python3 db_surrogate/train_gamma_surrogate.py \
--db gyrokinetic_simulations.db \
--name gamma_v1
```

## 4.5 Registering and using a trained surrogate

Model files alone are sufficient for offline prediction, but Datamak also supports database-backed registration through `gk_surrogate`. The GUI code contains helper logic to create that table if needed and record metadata for a trained model.

Once a `gk_surrogate` row exists, the script `db_surrogate/estimate_gamma_surrogate.py` can:

1. load the registered model,
2. reconstruct the same feature vector for all complete `gk_input` rows,
3. predict mean values and an uncertainty proxy,
4. upsert the results into `sg_estimate`.

## 5 End-to-End Summary

The complete operational chain is:

1. create or refresh the schema,
2. ingest equilibria into `data_equil`,
3. create active studies and templates,
4. generate `gk_input.content` with Pyrokinetics,
5. backfill structured plasma and geometry columns onto `gk_input`,
6. run selected inputs remotely and sync `gk_run` metrics,
7. train a surrogate from the joined `gk_input` and `gk_run` data,
8. optionally register the model in `gk_surrogate` and populate `sg_estimate`.

## 6 Practical Command Reference

```
python3 database/create_gyrokinetic_db.py --db gyrokinetic_simulations.db

python3 db_update/populate_data_equil_from_Mate_KinEFIT.py --db gyrokinetic_simulations.db

python3 db_update/create_gk_input_from_pyrokinetic_with_pfile_and_gfile.py \
    --db gyrokinetic_simulations.db

python3 db_update/backfill_gk_input_physics.py --db gyrokinetic_simulations.db

python3 batch/create_batch_database.py --copy-torun

python3 batch/deploy_batch_large.py

python3 db_surrogate/train_gamma_surrogate.py --db gyrokinetic_simulations.db --name gamma_v1

python3 db_surrogate/estimate_gamma_surrogate.py --db gyrokinetic_simulations.db --surrogate-id
```

## 7 Current Limitations

- The base schema bootstrap does not yet create `gk_surrogate`; that table is still created lazily in the GUI path.
- The exact entry point for `gk_study` creation depends on which ingestion branch is used.
- Some Flux full-auto scripts still carry environment assumptions that should be documented and validated more explicitly.